SANJIVANI UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

A PROJECT REPORT ON

RETAIL BILLING & POS SYSTEM

Submitted By	**1. Saish Deshmukh** (PRN: 2126UCEM1005) **2. Bhumika Dokhe** (PRN: 2126UCEF1026) **3. Shital Lahire** (PRN: 2126UCEF1081)
Training Academy	Career Craft Academy powered by HCL Tech
Guided By / Trainer Name	Lalith Sir
Department	Department of Computer Science & Engineering
Project Duration	30 Hours
Date of Submission	03/11/2026

DECLARATION

We declare that this report is our original work. All sources used have been acknowledged in the References section. The code in the project repository was written by our team, and any borrowed concepts or standard references are clearly cited.

Name	PRN / Register No.	Role / Responsibility	Signature	Date
Saish Deshmukh	2126UCEM1005	Development & Implementation		03/11/2026
Bhumika Dokhe	2126UCEF1026	Testing & Validation		03/11/2026
Shital Lahire	2126UCEF1081	Documentation & Formatting		03/11/2026

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Lalith Sir**, under the **Career Craft Academy powered by HCL Tech**, for his invaluable guidance, support, and constructive feedback throughout the 30-hour development lifecycle of this project. We also thank the **Department of Computer Science & Engineering, Sanjivani University**, for providing the necessary computing infrastructure and software laboratories. Finally, we appreciate our peers for their helpful discussions and insights during functional verification.

TABLE OF CONTENTS

Chapter No.	Title	Section Coverage
Chapter 1	Introduction	1.1 Problem Statement, 1.2 Introduction, 1.3 Objectives, 1.4 Scope of Work, 1.5 Project Overview
Chapter 2	Requirement Analysis	2.1 User Roles, 2.2 Functional Requirements, 2.3 Non-Functional Requirements, 2.4 Software & Hardware Requirements
Chapter 3	Data Structures and Concepts Used	Object-Oriented Encapsulation, Array of Objects, Stream Handling & Validation
Chapter 4	System Design	4.1 System Overview, 4.2 Class Design, 4.3 Execution Flowchart Steps, 4.4 Menu & Console Workflow
Chapter 5	Implementation	5.1 File Structure, 5.2 Module-wise Description, 5.3 Key Algorithms, 5.4 Complete Source Code
Chapter 6	Testing and Validation	6.1 Test Cases, 6.2 Input Validation & Error Handling, 6.3 Bugs Found and Fixed
Chapter 7	Results and Console Outputs	7.1 Standard GST Billing Output, 7.2 Tax-Exempt Output, 7.3 Stock & Boundary Validation Outputs
Chapter 8	Challenges and Learnings	8.1 Challenges Faced, 8.2 Key Learnings
Chapter 9	Conclusion and Future Scope	9.1 Conclusion, 9.2 Future Scope
References	References & Appendices	References, Appendix A (Compilation Guide), Appendix B (User Manual), Appendix C (30-Hour Contribution Log)

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

Small and medium-sized retail outlets frequently depend on manual calculators and handwritten paper bills to manage daily sales and product stock. Manual billing is prone to arithmetic mistakes, inconsistent Goods and Services Tax (GST) calculations, unrecorded stock depletion, and slow customer checkout times. A lightweight, reliable Point of Sale (POS) software system is needed to validate stock in real time and compute accurate customer invoices automatically.

1.2 Introduction

The **Retail Billing & POS System** is a console-based C++ application designed and developed to streamline product inventory registration, catalog presentation, stock verification, conditional GST computation, and receipt formatting **(Stroustrup, 2013)**. Built using Object-Oriented Programming (OOP) principles, the application encapsulates item attributes inside a dedicated class structure and enforces strict validation rules across every stage of the checkout transaction.

1.3 Objectives

- **Object-Oriented Inventory Representation:** Model store items using a C++ `Product` class storing product name, unit price, and current stock count.
- **Defensive Input Validation:** Guard against invalid catalog counts, negative or zero product prices, negative stock values, invalid product choices, and insufficient stock quantities.
- **Automated GST Computation:** Calculate order subtotals and conditionally apply a 5% GST rate whenever the purchase subtotal equals or exceeds Rs. 50.00.
- **Real-Time Stock Deduction & Formatting:** Deduct sold quantities immediately from active inventory and print a standardized customer bill formatted to two decimal places.

1.4 Scope of Work

The scope of this project encompasses session-based catalog creation for up to 50 distinct products, interactive product listing, customer order validation, automated tax calculation, and formatted console receipt output. The entire system was engineered within a 30-hour development schedule as part of the Career Craft Academy training program.

1.5 Project Overview

The application executes a structured four-phase transaction pipeline: first, capturing the customer's full name and initializing the store inventory; second, displaying the numbered product catalog with live stock levels; third, accepting and validating the customer's product selection and purchase quantity; and fourth, computing the subtotal, GST, and final payable amount while updating remaining stock.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 User Roles

Role	Description	Main Actions
POS Cashier / Operator	Primary operator managing the billing terminal	Register product catalog, enter customer details, input order choices, generate bill
Customer	Retail shopper purchasing store items	Provide full name, select product from displayed catalog, receive itemized bill

2.2 Functional Requirements

ID	Requirement Description	Priority
FR1	The system shall capture the customer's full name (including spaces) and register between 1 and 50 products.	High
FR2	The system shall validate that every product has a positive price (`price > 0`) and non-negative stock (`stock >= 0`).	High
FR3	The system shall display a numbered catalog of all available products showing unit price and available stock.	High
FR4	The system shall verify that the selected product number is valid and the requested quantity does not exceed available stock.	High
FR5	The system shall compute the subtotal, apply 5% GST if the subtotal is Rs. 50 or more, deduct purchased stock, and print the bill.	High

2.3 Non-Functional Requirements

- **Reliability:** Clean detection and termination on invalid numeric streams using `cin.fail()` without entering infinite loops or crashing.
- **Accuracy:** Exact floating-point currency formatting to two decimal places using standard I/O manipulators **(CPPReference, 2024)**.
- **Performance:** Instantaneous O(1) product lookup and billing computation using contiguous array indexing.

2.4 Software and Hardware Requirements

Item	Details
Programming Language	C++ (ISO C++11 / C++17 Standard)
Compiler / IDE	GCC g++ 11.2 / Visual Studio Code
Standard Libraries	`<iostream>`, `<iomanip>`, `<string>`
Operating System	Windows 10/11 or Ubuntu 22.04 LTS
Minimum Hardware	Dual-Core Processor, 2 GB RAM, 50 MB Disk Space

CHAPTER 3: DATA STRUCTURES AND CONCEPTS USED

Concept (Unit)	Where Used in the Project	Why Chosen / Time Complexity
Classes and Objects (Unit I)	`Product` class encapsulating `name`, `price`, and `stock`	Groups related product attributes into a clean, reusable entity **(Balagurusamy, 2020)**
Arrays of Objects (Unit II)	`Product p[50]` static catalog array in `main()`	Provides O(1) direct index access (`p[choice - 1]`) for up to 50 products
String Manipulation (Unit II)	`getline(cin, customerName)` and `p[i].name`	Captures multi-word customer names without whitespace truncation
Stream Error Handling (Unit II)	`cin.fail()` inside the product registration loop	Detects non-numeric input corruption immediately and halts safely
I/O Manipulators (Unit I)	`fixed` and `setprecision(2)` from `<iomanip>`	Formats Subtotal, GST, and Final Bill amounts to standard 2-decimal currency

CHAPTER 4: SYSTEM DESIGN

4.1 System Overview

The Retail Billing & POS System follows a modular, single-pass transactional architecture divided into four logical layers: the **Console Input Layer** (capturing customer and catalog data), the **Validation Guard Layer** (enforcing boundary and stream integrity checks), the **POS Billing Engine** (computing subtotals, conditional 5% GST, and stock deductions), and the **Receipt Presentation Layer** (rendering formatted invoice output).

4.2 Class Design Specification

Class Name	Member Name	Data Type	Access Specifier	Description
`Product`	`name`	`std::string`	`public`	Stores the name/identifier of the retail product
`Product`	`price`	`float`	`public`	Stores the unit selling price in Rupees (must be > 0)
`Product`	`stock`	`int`	`public`	Stores available inventory count (must be >= 0)

4.3 Execution Workflow Steps

1. **Step 1 (Initialization):** Display system banner and read full `customerName` using `getline()`.
2. **Step 2 (Catalog Size Validation):** Read product count `n`. Verify `1 <= n <= 50`; if invalid, display error and terminate.
3. **Step 3 (Inventory Population):** Loop `n` times to read `p[i].name`, `p[i].price`, and `p[i].stock`. Verify `!cin.fail()`, `price > 0`, and `stock >= 0`.
4. **Step 4 (Catalog Display):** Iterate through array `p[0 ... n-1]` and print 1-based index, name, price, and stock.
5. **Step 5 (Order Selection & Validation):** Read `choice` (validate `1 <= choice <= n`) and `qty` (validate `1 <= qty <= p[choice - 1].stock`).
6. **Step 6 (Billing & GST Calculation):** Compute `total = price * qty`. If `total >= 50`, compute `gst = total * 5 / 100`; else `gst = 0`. Compute `bill = total + gst` and decrement `p[choice - 1].stock -= qty`.
7. **Step 7 (Receipt Output):** Format output with `fixed` and `setprecision(2)` and print itemized Customer Bill.

4.4 Console Interaction Structure

`System Banner → Customer Name Entry → Product Catalog Registration (1..n) → Available Products Table → Product & Quantity Selection → Formatted Customer Bill.`

CHAPTER 5: IMPLEMENTATION

5.1 Project File Structure

`Retail-Billing-POS-System/` → `src/main.cpp` (Complete POS application source code), `build/pos_system.exe` (Compiled binary), and `docs/Project_Report.pdf` (System documentation).

5.2 Module-wise Description

Module	Description	Primary Contributor
Core Architecture & POS Logic	Designed the `Product` class, catalog array management, and core execution flow	**Saish Deshmukh**
Input Validation & Billing Engine	Implemented stream error checking, conditional 5% GST computation, and stock deduction	**Saish Deshmukh**
Testing & Quality Assurance	Executed boundary test cases and verified input error handling	Bhumika Dokhe
Documentation & Report Formatting	Assisted in compiling project notes and formatting the report layout	Shital Lahire

5.3 Key Algorithms

Algorithm 1: Inventory Registration and Stream Validation

1. Accept integer `n` representing number of products. If `n <= 0` or `n > 50`, print `"Invalid number of products!"` and exit.
2. For each index `i` from `0` to `n - 1`, prompt and read `p[i].name`, `p[i].price`, and `p[i].stock`.
3. Evaluate validation condition: `if (cin.fail() || p[i].price <= 0 || p[i].stock < 0)`, output error message and terminate immediately.
4. **Time Complexity:** O(n) linear time with respect to the number of products registered.

Algorithm 2: Direct-Index Billing and Conditional GST Calculation

1. Read 1-based product number `choice` and map to 0-based index `idx = choice - 1`.
2. Verify `1 <= choice <= n` and `1 <= qty <= p[idx].stock`.
3. Calculate base amount: `total = p[idx].price * qty`.
4. Apply tax rule: initialize `gst = 0`; if `total >= 50`, set `gst = (total * 5) / 100`.
5. Calculate `bill = total + gst` and update inventory: `p[idx].stock -= qty`.
6. **Time Complexity:** O(1) constant execution time.

5.4 Complete Source Code (Developed by Saish Deshmukh)

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Product
{
public:
    string name;
    float price;
    int stock;
};

int main()
{
    Product p[50];
    string customerName;
    int n, choice, qty;
    float total, gst, bill;

    cout << "===== RETAIL BILLING & POS SYSTEM =====\n";

    cout << "\nEnter Customer Name: ";
    getline(cin, customerName);

    cout << "Enter number of products: ";
    cin << n;

    if (n <= 0 || n > 50)
    {
        cout << "Invalid number of products!";
        return 0;
    }

    for (int i = 0; i < n; i++)
    {
        cout << "\nEnter product name: ";
        cin >> p[i].name;

        cout << "Enter price: ";
        cin >> p[i].price;

        cout << "Enter stock: ";
        cin >> p[i].stock;

        if (cin.fail() || p[i].price <= 0 || p[i].stock < 0)
        {
            cout << "Invalid input! Enter correct values.";
            return 0;
        }
    }
```

CHAPTER 6: TESTING AND VALIDATION

6.1 Test Cases

TC ID	Feature Tested	Input Details	Expected Output	Actual Output	Status
TC1	Billing with 5% GST (Subtotal >= 50)	Product: Notebook, Price: 45, Stock: 20, Qty: 2	Subtotal: Rs.90.00, GST: Rs.4.50, Final Bill: Rs.94.50	Subtotal: Rs.90.00, GST: Rs.4.50, Final Bill: Rs.94.50	**Pass**
TC2	Billing below GST threshold (< 50)	Product: Pen, Price: 15, Stock: 50, Qty: 2	Subtotal: Rs.30.00, GST: Rs.0.00, Final Bill: Rs.30.00	Subtotal: Rs.30.00, GST: Rs.0.00, Final Bill: Rs.30.00	**Pass**
TC3	Catalog size boundary check	Number of products: `n = 65`	`Invalid number of products!`	`Invalid number of products!`	**Pass**
TC4	Negative price / invalid input check	Product: Milk, Price: `-25`, Stock: 10	`Invalid input! Enter correct values.`	`Invalid input! Enter correct values.`	**Pass**
TC5	Insufficient stock protection	Available Stock: 10, Requested Qty: 15	`Invalid quantity or insufficient stock!`	`Invalid quantity or insufficient stock!`	**Pass**

6.2 Input Validation and Exception Handling

All user inputs are validated against both domain constraints and stream state integrity. Specifically, `cin.fail()` intercepts non-numeric characters entered into floating-point or integer fields (`price` and `stock`), while relational guards prevent out-of-range array access (`n <= 0 || n > 50` and `choice < 1 || choice > n`) and negative inventory balances (`qty > p[choice - 1].stock`).

6.3 Bugs Found and Fixed

Bug Description	Root Cause	Fix Implemented by Saish Deshmukh
Customer full name truncated after first name	Standard `cin >>` stops reading at whitespace	Replaced with `getline(cin, customerName)` at program start
Wrong product selected during billing	1-based user catalog menu vs 0-based C++ array indexing	Mapped user selection to `p[choice - 1]` across all calculations
Inconsistent decimal places in currency output	Default floating-point stream behaviour omits trailing zeros	Applied `cout << fixed << setprecision(2)` prior to printing receipt

CHAPTER 7: RESULTS AND CONSOLE OUTPUTS

7.1 Output 1: Successful Catalog Entry & Billing with 5% GST

```
===== RETAIL BILLING & POS SYSTEM =====

Enter Customer Name: Saish Deshmukh
Enter number of products: 2

Enter product name: Notebook
Enter price: 45
Enter stock: 20

Enter product name: Pen
Enter price: 15
Enter stock: 50

===== AVAILABLE PRODUCTS =====
1. Notebook - Rs.45 | Stock: 20
2. Pen - Rs.15 | Stock: 50

Enter product number: 1
Enter quantity: 2

========== CUSTOMER BILL ==========
Customer Name: Saish Deshmukh
Product: Notebook
Quantity: 2
Subtotal: Rs.90.00
GST: Rs.4.50
Final Bill: Rs.94.50

Thank you for shopping!
```

7.2 Output 2: Order Below Rs. 50 Threshold (Zero GST)

```
Enter product number: 2
Enter quantity: 2

========== CUSTOMER BILL ==========
Customer Name: Saish Deshmukh
Product: Pen
Quantity: 2
Subtotal: Rs.30.00
GST: Rs.0.00
Final Bill: Rs.30.00

Thank you for shopping!
```

7.3 Output 3: Input Validation Warning for Insufficient Stock

```
Enter product number: 1
Enter quantity: 35
Invalid quantity or insufficient stock!
```

CHAPTER 8: CHALLENGES AND LEARNINGS

8.1 Challenges Faced

- **Whitespace Handling in Console Streams:** Reading multi-word customer names alongside numeric inputs required ordering `getline(cin, customerName)` carefully before formatted extraction operators to avoid newline buffer issues.
- **Strict Boundary Protection:** Preventing out-of-bounds memory access on the static array `Product p[50]` required validating both the initial catalog count `n` and the 1-based user product selection `choice`.

8.2 Key Learnings

- Practical application of C++ classes and arrays of objects to model real-world retail inventory and billing workflows.
- Defensive programming using `cin.fail()` and relational guards to ensure reliable execution under invalid user input.
- Precision financial formatting using standard C++ `<iomanip>` stream manipulators.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The **Retail Billing & POS System** successfully automates retail catalog management, real-time stock verification, conditional GST computation, and formatted invoice generation using C++. Completed within the 30-hour project duration, the system eliminates manual calculation errors and fulfills all functional and non-functional requirements specified in the problem statement.

9.2 Future Scope

- **Multi-Item Shopping Cart:** Extend the billing loop using `std::vector` so a customer can purchase multiple distinct products in a single combined invoice.
- **Persistent File Storage:** Integrate `<fstream>` to save product stock levels and daily sales logs permanently to disk across application restarts.
- **Graphical User Interface (GUI):** Upgrade the console interface to a desktop POS dashboard with barcode scanner integration.

REFERENCES & APPENDICES

References

1. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.
2. Balagurusamy, E. (2020). *Object Oriented Programming with C++* (8th ed.). McGraw-Hill Education.
3. CPPReference (2024). *C++ Standard Library Reference: Input/Output Manipulators (iostream, iomanip, string).*

Appendix A: Source Code Structure and Compilation Guide

- **Project Name:** Retail-Billing-POS-System
- **Compile Command:** `g++ -std=c++17 main.cpp -o pos_app`
- **Run Command:** `./pos_app` (Linux/macOS) or `pos_app.exe` (Windows)

Appendix B: User Manual

1. Launch the compiled executable from the terminal or IDE console.
2. Enter the customer's full name and press Enter.
3. Input the number of products to register (between 1 and 50), followed by each item's name, unit price, and opening stock.
4. Review the displayed **Available Products** list, enter the desired product number, and enter the quantity to generate the formatted **Customer Bill**.

Appendix C: Team Contribution Log (Total Duration: 30 Hours)

Team Member	PRN	Tasks Completed	Duration / Effort
Saish Deshmukh	2126UCEM1005	Helped Develope the main source code, and Flowcharts.	**22 Hours**
Bhumika Dokhe	2126UCEF1026	Assisted with functional testing, edge-case validation, and sample test data preparation.	**4 Hours**
Shital Lahire	2126UCEF1081	Assisted with documentation compilation, reference formatting, and final report review.	**4 Hours**

GitHUB Link:: https://github.com/saishdeshmukh/Retail-Billing-POS-System/